

HelixQA

HelixQA

Anti-blöf QA orkestrasyonu — her GEÇER sonucunun, gerçek bir kullanıcının özelliği kullanabildiğine dair yakalanmış kanıt taşıdığı, otonom ve çapraz platform oturumları.

Özet

HelixQA, çapraz platform testleri (Android, Android TV, Web, Masaüstü) için geliştirilmiş bir anti-blöf QA orkestrasyon çerçevesidir. YAML test bankalarını, gerçek zamanlı çökme tespitini, adım adım kanıt yakalamayı ve LLM-artı-bilgisayarla-görü otonom QA oturumlarını bir araya getirerek özelliklerin gerçekten uçtan uca çalıştığını kanıtlar. Constitution'un zorunlu QA test türüdür (§11.4.169).

Kısa açıklama

Yazılı test bankalarını ve tam otonom, LLM ve görüntü işleme tabanlı QA oturumlarını platformlar arasında çalıştıran bir anti-blöf QA orkestratörü (Go). Çökmeleri tespit eder, her adımı yakalanmış kanıtlarla (ekran görüntüleri, logcat, video, yığın izleri) doğrular ve AI düzeltme hatları için kanıt zengini biletler otomatik olarak oluşturur.

Uzun açıklama

HelixQA, tek ve tavizsiz tasarım odağı Constitution'un §11.4 Operatif Kuralı olan bir Go çerçevesidir: yayınlama kriteri "testler geçti" değil, "kullanıcılar özelliği kullanabiliyor"dur. Bu nedenle, her GEÇER sonucu, yürütme sırasında yakalanmış pozitif çalışma zamanı kanıtı taşımalıdır — kanıt yoksa yeşil ışık yok, istisna yok. Hem yazılı hem de bilinmeyen durumları kapsayan iki tamamlayıcı modda çalışır. İlk olarak, **yazılı test bankaları** — platform hedeflemesi, öncelik, sıralı adımlar (ad/eylem/beklenen), etiketler

ve dokümantasyon referansları içeren TC-XXX vakalarından oluşan YAML paketleri — adım adım doğrulama, gerçek zamanlı çökme/ ANR tespiti (Android için ADB, web/masaüstü için süreç izleme), merkezi kanıt toplama ve AI düzeltme hatları için otomatik olarak oluşturulmuş Markdown biletleri ile yürütülür. İkinci olarak, **tam otonom QA oturumu** uygulamayı LLM destekli ajanlara ve bilgisayarla görüye devreder ve dört disiplinli aşamada kontrolsüz olarak çalıştırır: kurulum (LLM’lerin seçimi, proje belgelerinden özellik haritasının oluşturulması, CLI ajanlarının başlatılması, görüntü işleme motorunun hazırlanması), her belgelenmiş özelliğin dolaşıldığı belge odaklı doğrulama, kenar durumları ve belgelenmemiş davranışları bilinçli olarak test eden merak odaklı keşif, ardından her bulgunun video zaman damgalı kanıtlarla ilişkilendirildiği Markdown/HTML/JSON formatında raporlama ve temizlik.

En kritik nokta, kendi ödevini notlandırmamasıdır: Dört harici Go alt modülünü (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) entegre eder ve ortak challenges ile containers altyapısını kullanır. Böylece uygulamayı gezen bileşen, çalışıp çalışmadığını değerlendiren bileşenle aynı değildir. Kendi paketi, make anti-bluff (statik tarama + davranış çapa manifestosu + mutasyon mandalı) ve yerleşik §1.1 mutasyonuna sahip 8 aşamalı bir orkestratör Challenge ile başkalarına uyguladığı aynı kriterlere tabi tutulur. 15 satırlık test türü kapsama matrisi, her ilan edilen yeteneği somut bir yürütülebilir varlığa ve belirli bir yakalanmış kanıt biçimine bağlar — böylece çerçevenin kendisi hakkındaki iddiaları, test ettiği ürünlere verdiği kararlar kadar kanıta dayalıdır.

İçerik

Neden inşa ettik

Geleneksel Kalite Güvencesi (QA), “doğrulama başarılı” uyarısını yeşil ışık olarak kabul eder. İşte tam da bu yüzden Constitution’un *blöf* olarak adlandırdığı başarısızlık sınıfı gözden kaçır: kullanıcıya göre çalışmayan bir özellik, raporlarda sorunsuz görünür. HelixQA, özellikle QA için bu durumu imkânsız kılmak üzere tasarlandı. Bir doğrulamanın BAŞARILI olarak değerlendirilmesi için gerçek çalışma sırasında elde edilmiş fiziksel kanıt (ekran görüntüsü, logcat, video, yığın izi, rapor) gerektirir ve kanıt olmadan verilen yeşil özet satırını, eksik bir özellik eşdeğer kritik bir hata olarak kabul eder. Ayrıca, birçok platformda kapsamlı manuel QA’nın ölçeklenememe sorununu da çözer: oturumları tamamen otonom hale getirerek.

Neden oyunun kurallarını deęiřtiriyor

Neredeyse hibir arata bir arada bulunmayan iki unsuru birleřtiriyor: titiz, kanıta dayalı QA geiř kontrolü ve otonom, kendi kendine alıřan keřif. Bir LLM artı grsel algılama aracısı, *gerek* uygulamayı aar, belgelenmiř her zellięi doęrular, kimsenin test yazmadıęı belgelenmemiř hataları avlar ve bunu yaparken mahkeme kalitesinde bir kanıt zinciri retir. Bylece “test ettik” ifadesi, “iřte video, iřte logcat, iřte bilet” haline gelir. Ayrıca Constitution tarafından adlandırılan QA alt modl olduęu iin, benimsenmesi tek bir ekibin QA drstlęn artırmakla kalmaz; aynı hamlede tm rn ailesindeki kalite tabanını ykseltir.

Yeniliki ynleri

- **Blf karřıtı kanıt szleřmesi** – Her kontroln BAřARILI sonucu, alıřma zamanı sırasında elde edilmiř kanıtlarla baęlanır; yeřil bir CI satırı gerekli grlse de asla yeterli kabul edilmez ve kanıtsız yeřil zet, kritik bir hata olarak deęerlendirilir.
- **Otonom belge odaklı + merak odaklı keřif** – Belgelenmiř her zellięi doęrular, ardından senaryo dıřına ıkararak gerek kullanıcıların karřılařtıęı sınır durumları (boř giriřler, hızlı etkileřimler, belgelenmemiř yollar) arařtırır; hibir elle yazılmıř test paketinin ngremeyeceęi durumları tespit eder.
- **Grsel kehanet** – GoCV mekanik grnt iřleme ve LLM Grsel API, alıřan arayz *gerekten grerek*, token ve zellik dzeyindeki doęrulamaların gzden kaırdıęı grsel hataları yakalar.
- **Yapıya dayalı, metne dayalı olmayan test bankaları** – Banka dizeleri yapıyı tanımlar ve alıřma zamanında LLM tarafından oluřturulan soru istemlerini ynlendirir (CONST-046). Bylece tek bir banka, arayz metni evrildięinde bile farklı yerelerde alıřmaya devam eder.
- **AI onarım hatları iin hazır biletler** – Otomatik oluřturulan Markdown sorunları, tam kanıt paketiyle birlikte gelir ve bir insan triyajcısı yerine doęrudan bir onarım aracısına iletmeye hazırdır.

Tüm ürünlerde nasıl kullanılıyor (sunduğu güçler)

Zorunlu kalite sütunu olarak (Constitution §11.4.169, helix_qa alt modülünü gerekli test türlerinden biri olarak adlandırır), HelixQA ailesindeki her ürüne aynı güçleri kazandırır:

- **Otonom QA oturumları:** Tek bir `helixqa autonomous --project ... --platforms android,desktop,web` komutu, gerçek uygulamaları hedef bir kapsama alanına yönlendiren, rapor, bilet ve videolar üreten bir LLM artı görsel algılama aracısını serbest bırakır; süreçte insan müdahalesine gerek kalmaz.
- **Test bankaları / paketleri:** YAML bankaları (en az 30 ile 219. tur tabanı), platforma özel, öncelik sıralı ve doğruladıkları belgelerle satır satır izlenebilir.
- **Kaydedilmiş kanıtlar:** Ekran görüntüleri, logcat, video, yığın izleri ve tam zaman çizelgesi – her rapordan merkezi olarak erişilebilir ve bağlanabilir, böylece her karar sonradan yeniden izlenebilir ve denetlenebilir.
- **Bağımsız kararlar (§11.4.141 bağımsızlık ilkesi):** LLM destekli issuedetector ve görsel kehanet aracısı, uygulamayı gezen araçıdan bağımsız olarak çalışan uygulamanın davranışını değerlendirir; böylece bir sistemin kendi işini doğru olarak işaretlemesi klasik hatası yapısal olarak engellenir.
- **Geçiş kontrolü + mutasyon kilidi:** `make qa-all / make anti-bluff` ve `challenges/scripts/helixqa_orchestrator_challenge.sh` (8 aşamalı, §1.1 yerleşik mutasyon) komutları, HelixQA'un kendi dürüstlüğüne sürekli olarak kanıtlamasını sağlar. Ayrıca, son teslim tarihleri altında disiplini devre dışı bırakmak için kasıtlı olarak `--skip-helixqa` gibi bir kaçış yolu bırakılmamıştır.

İçerik

En Büyük Teknik Zorluklar ve Bunları Nasıl Çözdük

- **QA sürecindeki yanlış pozitiflerin önlenmesi** — blöf yakalayan araç, kendisi blöfe dönüşmemeli → her adım, yakalanan kanıtlara karşı doğrulanır; kanıt olmadan GEÇER notu, GEÇER yerine hata olarak puanlanır; davranış-çapa manifestosu, her ilan edilen yeteneği çalıştırılabilir bir teste bağlar (CONST-035), böylece bir yetenek, onu uygulayan bir şey olmadan iddia edilemez.
- **Tek bir beyinle heterojen platformların yönetilmesi** — Android, Android TV, Web ve Masaüstü'nün ortak bir giriş modeli yok → tek bir navigator paketi, platforma özel ActionExecutor'ları (ADB, Playwright, X11) ve platforma özgü

çökme dedektörlerini (android/web/masaüstü) soyutlar; böylece orkestrasyon mantığı tek sefer yazılır ve platform farklılıkları kenarlarda kalır.

- **Otonom ajanların kaotik değil, kullanışlı hale getirilmesi** — denetimsiz bir LLM, bir uygulamada sonsuza dek dolaşabilir → LLMsVerifier doğru modelleri puanlar ve seçer, LLMOrchestrator başsız CLI ajanlarını (opencode, claude-code, gemini, junie, qwen-code) yönetir, DocProcessor keşfe hedef veren özellik haritasını oluşturur ve VisionEngine her kararı, modelin hayal gücü yerine ekrandaki gerçek piksellere dayandırır.
- **Yerelleştirme güvenli test bankaları** — İngilizce UI metnini sabit kodlayan bir test paketi, on beş dilde bozulur → bankalar yalnızca yapıyı tanımlar ve kullanıcıya yönelik metinler, çalışma zamanında LLM/kaynak yüklenir (CONST-046); böylece aynı banka, yerel ayardan bağımsız olarak aynı davranışı doğrular.
- **Kapıların sahte olmadığının kanıtlanması** — kendisi başarısız olamayan bir blöf önleme kapısı, nihai blöftür → eşleştirilmiş §1.1 mutasyonları, bir türün kanıt toplama veya blöf önleme iddiasını kaldırır ve kapının BAŞARISIZ olmasını zorunlu kılar; zaman içinde bu garantinin sessizce aşınmasını önleyen bir mutasyon mandalı da devreye girer.

Teknoloji Yığını

- **Go 1.24+ orkestratörü** — *neden*: QA, ürünlerin çalıştığı her yerde çalışmalı; bu nedenle tek bir statik olarak bağlanmış, hızlı ve taşınabilir ikili, çalışma zamanı ağır bir alternatife tercih edilir; *nasıl*: cmd/helixqa CLI, birleştirilebilir alt komutlar sunar: run / list / report / autonomous / version.
- **YAML test bankaları (pkg/testbank)** — *neden*: test paketleri bildirimsel ve okunabilir olmalı, insanlar Go'a dokunmadan düzenleyebilmeli; *nasıl*: version/name/test_cases[] yapısı, id, category, priority, platforms, sıralı steps[] ve documentation_refs[] içerir; bu sayede özellik belgelerine izlenebilirlik sağlanır.
- **Çökme/ANR dedektörleri (pkg/detector)** — *neden*: en önemli hatalar, etkileşim sırasında canlı olarak meydana gelenlerdir, sonradan yapılan bir doğrulama değil; *nasıl*: Android için ADB (pidof/logcat/screencap), web/masaüstü için pgrep kullanılır; test çalışırken süreç izlenir.
- **Kanıt toplama (pkg/evidence, pkg/session)** — *neden*: blöf önleme sözleşmesi, ancak her GEÇER notunun fiziksel kanıtlarla desteklenmesi halinde gerçektir; *nasıl*: ekran görüntüleri, logcat, video ve yığın izleri, her raporun geri bağlandığı bir SessionRecorder zaman çizelgesine kaydedilir.
- **Otonom oturum (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — *neden*: dört platformda kapsamlı manuel QA ölçeklenemez; bu nedenle keşfin kendisi otonom olmalı; *nasıl*:

4 aşamalı bir SessionCoordinator, ActionExecutor'lar (ADB/Playwright/X11) ve görsel, UX, erişilebilirlik ve işlevsel hataları kapsayan LLM hata tespiti.

- **Harici alt modüller** — *neden*: yeniden kullanım ve bağımsızlık (CONST-051), ayrıca — kritik olarak — gezgin ile değerlendiricinin ayrılması; *nasıl*: LLMsVerifier (model puanlama), LLMOrchestrator (başsız CLI ajanları), VisionEngine (GoCV + LLM Vision), DocProcessor (özellik haritası/kapsam), her biri bağımsız olarak sahiplenilen bileşenler.
- **Blöf önleme kapıları + mutasyon mandalı** — *neden*: HelixQA'un, §1.1 sözleşmesini her şeyde uyguladığı gibi kendisine de uygulanmasını sağlamak; *nasıl*: make anti-bluff taraması, bir davranış-çapa manifestosu ve mutasyon mandalı, helixqa_orchestrator_challenge.sh ile 8 aşamalı bir uçtan uca doğrulayıcı.
- **15 satırlık kapsam matrisi (docs/test-coverage.md)** — *neden*: CONST-050(B), hiçbir boşluk bırakmayan, tamamen hesaplanabilir bir test türü kümesi zorunlu kılar; *nasıl*: her satır, somut bir çalıştırılabilir varlığa ve belirli bir kanıt toplama biçimine bağlanır; böylece kapsam, bir iddia değil, kontrol edilen bir gerçek haline gelir.

İçerik

Durum ve dürüstlük notları

- **Durum: beta.** Aktif olarak geliştirilmekte (README durum afişi 219. tur). Kendi “anti-blöf” çubuğuna tabi tutuluyor.
- **Lisans: Apache-2.0.** Kurulum: go install digital.vasic.helixqa/cmd/helixqa@latest.

Öncelik katmanı: Helix-birincil — Helix ailesinin özelliklerin gerçekten çalıştığını doğrulama yönteminin zorunlu kalite/anti-blöf dayanağı.